Will this work?:
```
shopt -q globstar
GLOBSTAR_STATE=$?
shopt -q nullglob
NULLGLOB_STATE=$?
trap '
  unset -f usage
  if [ $NULLGLO
```

Will this work?

Mostly yes, but there are important caveats. Let me break it down.

What it does

1. Captures the current state of `globstar` and `nullglob` into variables (`0` = set, `1` = unset).
2. Installs a `RETURN` trap that restores both options and cleans up.

The `RETURN` trap gotcha

`RETURN` fires when a **function** (or `.` -sourced script) returns — **not** when a script itself exits. So this trap only restores state if:

- The code is inside a **function**, and
- That function actually returns while the trap is installed.

If you're at top level of a script, **the trap will never fire** — your options will leak and your `unset` s won't happen.

Also, `RETURN` traps can be inherited by nested functions depending on `functrace` , which can cause surprises (the trap could fire earlier than you expect and reset your options mid-run).

Bugs / issues

1. Logic is inverted by `shopt -q` semantics — actually it's fine, but

confusing

`shopt -q foo` returns `0` if **set**. Your code:

```bash
if [ $NULLGLOB_STATE -eq 0 ]; then shopt -s nullglob   # was set → re-set ✓
else shopt -u nullglob; fi                              # was unset → unset ✓
```

That's correct. Just note it's a roundabout way to write `shopt -s/-u .`

2. The trap string is single-quoted — variable expansion happens at trap-fire time

That's what you want here, but it means `NULLGLOB_STATE` / `GLOBSTAR_STATE` must still exist when the trap fires. You `unset` them *inside* the trap **after** using them — fine. But if the trap fires **twice** (nested function return, `functrace` on), the second run sees them unset → `[ -eq 0 ]` errors with "integer expression expected" and may accidentally unset options.

3. `unset -f usage_msg allow_decorators / cleanup`

These look like they belong to some other snippet. If those functions don't exist, `unset -f` is harmless (no error), so this is just dead code / leftover.

4. `shopt -q` under `set -e`

If you have `set -e` active, `shopt -q` returning `1` (option unset) — which is the *common* case — will **exit your script**. You're capturing `$?` on the next line, so this is a real hazard:

```bash
shopt -q globstar      # returns 1 → set -e kills script here
GLOBSTAR_STATE=$?
```

You need `shopt -q globstar || true` or `if shopt -q globstar; then ...`.

Better version

```bash
_saved_globstar=$(shopt -p globstar)   # e.g. "shopt -u globstar" or "shopt -s globstar"
_saved_nullglob=$(shopt -p nullglob)

restore_shopt() {
    eval "$_saved_globstar"
    eval "$_saved_nullglob"
    unset _saved_globstar _saved_nullglob
    trap - RETURN
}

# Only install a RETURN trap if we're actually inside a function,
# otherwise use EXIT:
if [[ ${FUNCNAME[0]+set} && ${#FUNCNAME[@]} -gt 1 ]]; then
    trap restore_shopt RETURN
else
    trap restore_shopt EXIT
fi
```

`shopt -p name` prints the exact `shopt -s` / `shopt -u` command to reproduce the current state — much cleaner than storing `$?` and re-branching. And choosing `EXIT` vs `RETURN` based on context avoids the top-level no-op problem.

Bottom line

- **Works as written** if: the snippet lives in a function, no `set -e`, the trap fires exactly once, and you don't care about the stray `unset -f` lines.
- **Fragile** because of `RETURN`-at-top-level, `set -e` + `shopt -q`, and double-fire risk.
- **Use** `shopt -p` + `EXIT`-**fallback** for a robust version.